

I/O from Python to the operating system

The call you write and the call the operating system runs are not the same call. `fflush()` flushes only the C library's buffers, and the kernel buffers need a second flush [S3]. Three of the four boundaries below your code have a tool that watches them [V10] [S1] [V30]. No tool in this lecture watches the fourth [V5].

Ondrej (Ondra) Krajicek

me@ondrejkrjicek.com · www.linkedin.com/in/OndrejKrajicek

Built 2026-09-14 08:27 UTC

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

Roadmap

1	What does one byte cost?	A stopwatch.
2	Where does the difference come from?	A tracer.
3	It is not one buffer — it is a stack	A diagram.
4	The same write, at three levels	A program.
5	Two operating systems, side by side	Two traces.
6	What we have not looked at	One open question.

Takeaways

- | | | |
|----------|--|--|
| 1 | Buffering pays on adjacency | A seek forces the buffer out. |
| 2 | A library call is not a system call | Several levels sit beneath yours [S3]. |
| 3 | The boundary is observable | A tracer shows what your line became [S1]. |
| 4 | The platforms differ by design | Each nominates a different level [V8]. |

SCOPE

What this lecture covers

- 1 Why two programs differ
- 2 When a buffer helps
- 3 The four levels
- 4 Reading a trace

SCOPE

What this lecture leaves out

- 5 When a write is safe
- 6 Memory mapping
- 7 Operating the tools
- 8 Asynchronous I/O

PART I

What does one byte cost?

Two programs. The same mebibyte. Predict the ratio before we run them.

What does one byte cost?

Two programs write the same mebibyte to a file. That is 1,048,576 bytes. One uses a buffer. One does not.

```
<h1>A: unbuffered – one call per byte</h1>
with open("a.out", "wb", buffering=0) as f:
    for i in range(1024 * 1024):
        f.write(b"x")

<h1>B: buffered – the default</h1>
with open("b.out", "wb") as f:
    for i in range(1024 * 1024):
        f.write(b"x")
```

One argument differs. Python documents it: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

[S6] Built-in Functions, open() — Python documentation. <https://docs.python.org/3/library/functions.html#open>

Two programs, two access patterns

Sequential access

Each write continues where the last one ended.

Random access

Each write follows a move to a new offset.

Why it matters

Lampson advised batch processing in 1983. He named sequential access in the next sentence, as a second and separate reason [S12].

[S12] Hints for Computer System Design — Lampson (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>

Predict before we run

Write down a ratio. How much slower is the unbuffered program?

1×

nothing worth measuring

10×

an optimisation

100×

a different kind of thing

Writing a number down before the run changes what you do with the result. Taber calls the method predict-observe-explain, and reports that "making a prediction will give students some sense of ownership in the following activity" [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

The measurement runs here, not on a slide

I print no ratio here, on purpose. A benchmark you did not watch me produce is the kind of claim this course teaches you to distrust.

Held fixed

Loop, interpreter, iteration count, bytes, filesystem.

Varied

One argument, `buffering=0` [S6].

Write down

Both times, the Python version, the buffer size.

What it includes

The operating system's own memory for file data, in both runs. Name the cache layer your number includes, because you cannot remove all of them [V19].

Your laptop will not agree with this projector

The buffer size is not a constant. It grew sixteen-fold in Python 3.14.

Python 3.13

The default "was set to 8192 since 16 years ago" [S7].

Python 3.14

"the buffer will typically be 128 kilobytes long" [S6].

Why

Write performance improved "3 to 5 times" over the 8192-byte default [S7]. The reporter measured it. No independent reproduction.

So

Report your version and buffer size with every number.

[S6] Python documentation, `open()`. [S7] CPython issue 117151. <https://github.com/python/cpython/issues/117151>

Where does the difference come from?

One answer, stated immediately, and then the instrument that proves it.

Where does the difference come from?

The difference is the number of times each program crosses into the operating system.

Your theory course defined a system call. This lecture counts them: each one makes a **crossing** out of your program, "a privilege transition, not a jump" [V15].

A	Unbuffered	One call per byte, so 1,048,576 calls.
B	Buffered	One mebibyte divided by the buffer size.
=	So measure first	At 8 KiB, 128 calls. At 128 KiB, 8 calls.

What one crossing costs

4,972

instructions, one stat call

13,585

cycles for the same call

2,559

L3 lines evicted — your data, not the kernel's

Soares and Stumm measured this on 2010 hardware [S16]. The counts are not current [V2]. What carries over is the regime distinction, not the numbers. That distinction is a call rate. The lost cache state dominates at one call per 2,000 to 100,000 instructions. The transition cost takes over above that rate [S16].

[S16] FlexSC — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf

What one trace line says

A trace records the system calls a program made, in order, with arguments and return values.

```
open("/dev/null", O_RDONLY) = 3
```

- 1 The name** Which system call ran [S1].
- 2 The arguments** In parentheses, decoded [S1].
- 3 The return value** After the equals sign [S1].

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did.

[S1] `strace(1)` — Linux man-pages, which documents this example line. <https://man7.org/linux/man-pages/man1/strace.1.html>

Count the crossings

The same two programs, run again under a counting tracer.

Command `strace -c -e trace=write python3 a.py` [S1]

Program A One write per byte.

Program B One write per buffer fill.

First Print `io.DEFAULT_BUFFER_SIZE`, then divide [S6].

Two runs A traced run cannot be timed [V17].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>

The count ratio over-predicts the time

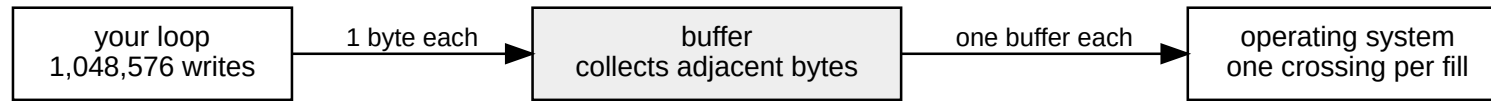
Illustration, not a measurement from this room: a count ratio of 100,000 beside a time ratio of 40 is not a mistake.

$\text{time} \approx \text{crossings} \times \text{fixed cost} + \text{bytes} \times \text{marginal cost [V1]}$

- 1 Two terms** Not a function of bytes alone [V1].
- 2 One term is shared** Both programs write the same mebibyte through the same loop.
- 3 So counts over-predict** Under this model the count ratio exceeds the time ratio.

What a buffer actually does

A buffer collects adjacent bytes and hands them down in one call.



Cost has two terms: the number of crossings, and the number of bytes [V1]. A buffer cuts the first and leaves the second alone, so the saving sits entirely in the call count.

Why a seek gives the saving back

A buffer holds bytes for consecutive positions. Move elsewhere and those bytes cannot stay in it.

- | | | |
|----------|--------------------|---|
| 1 | Constraint | Buffered bytes belong at the old offset. |
| 2 | Consequence | Every seek forces the buffer out. |
| 3 | Result | The buffered run converges on the unbuffered. |

The rule is **buffering is fast when writes are adjacent, and buys little when they are not**. Lampson gave the batching half in 1983 [S12].

Does the seek itself cost a crossing?

Yes. And a pair of system calls exists to remove it.

Naive

`lseek` then `write` — two crossings for one operation.

Merged

`pread` and `pwrite` "combine[s] file access and seek operations into a single command" [V3].

Measured

The merged pair cut the call count [V3]. For the same data volume the study also reports "a relatively reduced load in terms of overall duration" [V27].

Trust the counts. Do not trust the times.

The tracer stops your program twice per call. That destroys the timing and leaves the counting intact.

EVERY CALL STOPS	[V13]	So "the counts are trustworthy and the absolute times are not".	Counts only
GREGG'S DD LOOP	[V17]	One byte per call, "runs 442× slower under strace".	His machine
RE-RUNS	[V31]	"roughly 37× to 136×", other hardware.	Not a bound

The range does not contain Gregg's figure. Each number measures one machine and one workload.

Windows is absent here, on purpose

I used one Linux tool here and no Windows tool. Here is the reason, before you infer one.

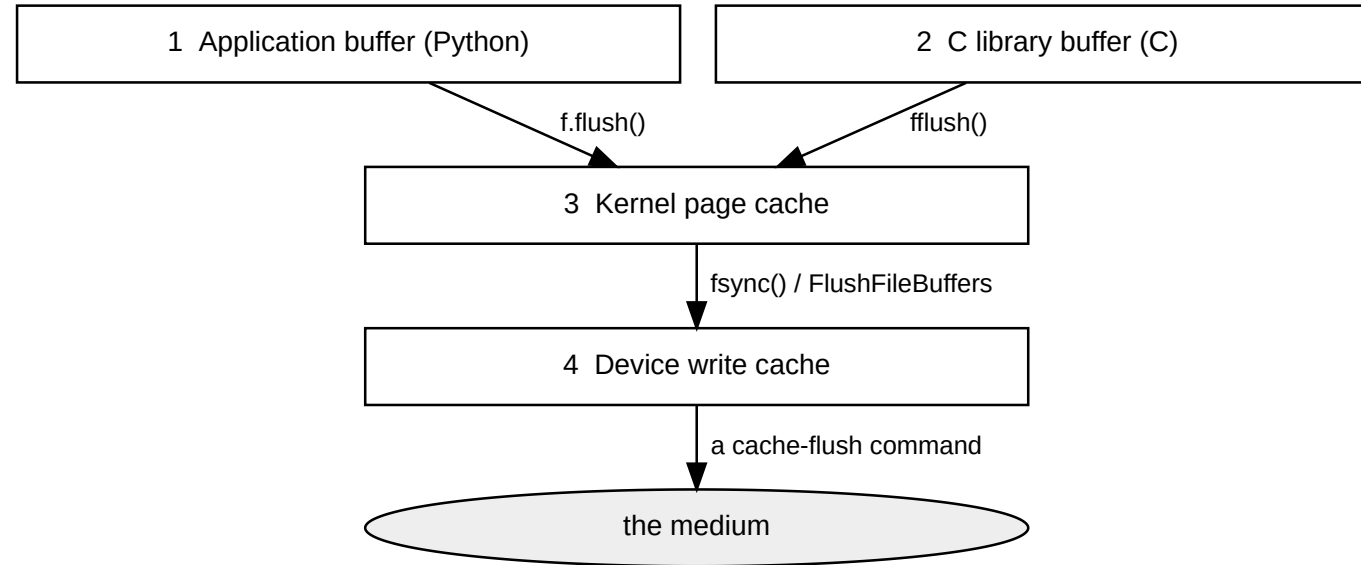
- | | | |
|----------|----------------------|---|
| 1 | Why | The Windows tracer has no counting summary to match. |
| 2 | Where it goes | Section five, with a real Windows trace [S10]. |
| 3 | The promise | Equal footing is a course promise, not a per-lecture one. |

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/>

It is not one buffer — it is a stack

We counted the crossings. Now we ask where the bytes actually are.

It is not one buffer — it is a stack



Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages. [S13] `fsync` — The Open Group.

Four levels, and who empties each one

LEVEL	WHO OWNS IT	EMPTIED BY
1 Application	the runtime's own buffer [S8]	<code>f.flush()</code> [S17]
2 C library	the <code>stdio</code> buffer [S3]	<code>fflush()</code> [S3]
3 Kernel	the page cache [V32]	<code>fsync()</code> [S13]
4 Device	the drive's write cache [V20]	a cache-flush command [V33]

The page cache lives in the machine's memory [V32]. It is not the disk's cache — the disk has its own, one level further down [V20].

Reads use the same four levels, in the other direction. The kernel fetches pages ahead of a sequential reader, so "a sequential `read()` just happens to return faster because the data was already fetched" [V34].

To flush is to cross one boundary

The verb means one thing here: to hand a buffer's contents to the next level down.

The C library manual says it plainly: "Note that fflush() flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3].



Do not say

Save, commit, persist — all claim durability.



Say

Flush. It names one boundary.

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

Wholesale down, retail up

Every level buys a large block from the level below. It sells small pieces to the level above.

That asymmetry is why every level in this diagram holds a buffer. Each level's "unit of purchase is larger than its unit of sale" [V18].

- | | | |
|----------|-----------------------------|-------------------------------------|
| 1 | Buys in blocks | 4096 bytes on an ext4 file [V7]. |
| 2 | Sells in bytes | Your loop writes one byte. |
| 3 | Same shape elsewhere | Allocators, caches, network stacks. |

Python's buffer does not sit on the C library's

Levels one and two are alternatives, not a pile.

INTUITIVE, AND WRONG

A Python write falls through the runtime buffer into the C library buffer and then into the operating system.

WHAT HAPPENS

"CPython reimplements the same policy above libc rather than inheriting it" [V6]. Python uses level one. C uses level two.

Python documents a typical **128 KiB** [S6]. The C library takes 4096 on an ext4 file [V7]. Two adjacent levels, two sizes, neither in your source.

[S6] Built-in Functions, `open()` — Python documentation. <https://docs.python.org/3/library/functions.html#open>

The bottom level acknowledges early too

Level four is the uncomfortable one, and skipping it would make the diagram dishonest.

- 1 The device holds a buffer** "most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache" [V20].
- 2 The standard allows a no-op** On fsync: "a null implementation is permitted" [S13].
- 3 Software cannot check** No trace separates a device that flushed from one that only reported it [V5].

[S13] fsync — The Open Group. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

Three lines that print nothing

A demonstration. You have the stack now, so predict this one.

```
import os  
print("hello")  
os._exit(0)
```

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Why the file is empty

The rule sits in the C standard, not in the kernel.

- 1 Mode follows destination**

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].
- 2 A terminal is interactive**

Bytes go out per line.
- 3 A file is not**

Bytes wait.
- 4 The exit skips cleanup**

No write reaches the kernel.

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

What this demonstration does not prove

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a kill -9. Killing the process loses only the *runtime's* buffer; the kernel still writes back." [V14]

- | | | |
|----------|-------------------------|---|
| 1 | What died | The process, and the level-one buffer with it. |
| 2 | What survived | Anything already in the page cache [V14]. |
| 3 | Wrong conclusion | "So flush() was enough. " It was enough against this test. |

Which levels can you actually watch?

I promised three boundaries of four. Here is the tool each one needs.

APP → LIBRARY	[V10]	"printf is visible to ltrace and write to strace".	Another tool
LIBRARY → KERNEL	[S1]	Where our tracer attaches.	Today
KERNEL → DEVICE	[V30]	Block-layer tracing, not in this course.	Elsewhere
DEVICE → MEDIUM	[V5]	No trace separates a flush from a report of one.	Never

The same write, at three levels

Now that the levels have a reason to exist, walk down them with one small program.

The same write, at three levels

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] `fflush(3)` — Linux man-pages. [S6] Python documentation, `open()`. [S20] `os` — Miscellaneous operating system interfaces, `os.write()`. <https://docs.python.org/3/library/os.html#os.write>

Meet the program: `iodemo.c`

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

```
f = fopen("no-such-file.txt", "rb"); /* 1. deliberate failure */
if (f != NULL) { fclose(f); return 2; }

f = fopen(path, "wb");                /* 2. the real file */
if (f == NULL) { return 1; }

fwrite("one\n", 1, 4, f);              /* 3. three writes */
fwrite("two\n", 1, 4, f);              /*    fourteen bytes */
fwrite("three\n", 1, 6, f);

if (fclose(f) != 0) { return 3; }     /* 4. flush, then close */
```

It prints nothing, on purpose. A silent program makes a legible trace.

Three calls in the source. One in the trace.

3

`fwrite` calls in the source

14

bytes written in total

1

`write` call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source:

```
write(3, "one\ntwo\nthree\n", 14) = 14
```

The 3 is the file descriptor. The 14 is all three `fwrite` calls, in one crossing.

[S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

`fwrite` takes a size and a count

Four arguments, and two of them multiply.

```
fwrite("three\n", 1, 6, f);  
/*      data      ^  ^  stream  
               |  +-- number of items  
               +----- size of one item, in bytes */
```

1 It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

2 Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

Two operating systems, side by side

Read as a comparison. Running the tools is the first lab's job.

Why NtTrace and not Process Monitor

Two Windows tools, two layers. One answers this lecture's question.

PROCESS MONITOR

- 1 Shows "real-time file system, Registry and process/thread activity" [S18]
- 2 The usual Windows choice
- 3 The wrong layer for us

NTTRACE

- 4 Reports native API calls, in user mode [S10]
- 5 Runs on the debugger interface, so it "needs no driver and no administrator rights" [V21]

[S18] Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Three Windows words you will need

Win32

The documented interface a Windows program calls. The 32 is history, not a width. Microsoft renamed it the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

The native API

The layer below Win32, inside `ntdll.dll`. Microsoft calls it "internal to the operating system" [S9].

The object manager

The kernel component that names open objects. Its paths begin `\\??\\` [V22].

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22]
Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

A real Windows trace

You can already read this. The anatomy matches the Linux line, and the vocabulary differs.

```
NtOpenFile(FileHandle=0x12fb38 [0x14], DesiredAccess=FILE_EXECUTE|SYNCHRONIZE,  
ObjectAttributes="\??\C:\WWW\NtTrace\ ", IoStatusBlock=0x0012FAE4 [0/1],  
ShareAccess=3, OpenOptions=0x21) ⇒ 0  
NtQueryAttributesFile(ObjectAttributes="\??\C:\WINDOWS\system32\cmd.exe.Local",  
Attributes=0x0012FADC [0]) ⇒ 0xc0000034 [2 'The system cannot find the file specified.']  
...  
NtWriteFile(FileHandle=4, Event=0, ApcRoutine=null, ApcContext=null,  
IoStatusBlock=0x0012FD8C [0/0x29], Buffer=0x4ad30e40, Length=0x29,  
ByteOffset=null, Key=null) ⇒ 0
```

Three lines excerpted from Roger Orr's published trace of `cmd` [S10]. Ignore every argument except the handle, the path and the return value.

[S10] NtTrace — Roger Orr, project page. <https://rogerorr.github.io/NtTrace/>

A handle is not a file descriptor

First difference, visible in the first argument of almost every line.

LINUX

A file descriptor. A small integer indexing a per-process table.
The trace showed = 3 [S1].

WINDOWS

A handle. An opaque value naming an open object. The trace shows `FileHandle=4` [S10].

Same job in a trace, different mechanism in the system. Do not use one word for the other.

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/>

Two error namespaces in one return value

Second difference, and it is the layer boundary showing up where nobody expects it.

```
⇒ 0xc0000034 [2 'The system cannot find the file specified.']
```

- | | | |
|----------|----------------------|---|
| 1 | First number | An NTSTATUS, the native layer's code [S10]. |
| 2 | Second number | The Win32 error it translates into [S10]. |
| 3 | Linux | One errno. Nothing to translate. |

[S10] NtTrace — Roger Orr, published sample trace. <https://rogerorr.github.io/NtTrace/>

The stable boundary sits at a different level

Third difference, and the other two are symptoms of it.

The Windows path reads "\\??\\C:\\WINDOWS\\system32\\cmd.exe.Local". That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

LINUX

[V8]

The system call interface is the public contract.

Syscall

WINDOWS

[S9]

Native entry points are "subject to change from one release of Windows to another".

Win32

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

Where the boundary sits is contested history

Two readings survive the evidence. I will not pick one for you.

- 1** **Reading A: deliberate** POSIX, OS/2 and Win32 each reached NT through the native API [S19].
- 2** **Reading B: vestige** A reported POSIX subsystem that was "never really completed" [S15].
- 3** **Linux had no choice** It "had to make the kernel boundary itself the stable interface" [V38].

[S15] The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] Inside Native Applications — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

Linux is the outlier, not Windows

The tempting conclusion is backwards, and the evidence is a list of other systems.

"Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9]

- | | | |
|----------|---------------------------------|---|
| 1 | Who declines the promise | FreeBSD, OpenBSD, NetBSD, illumos, macOS — and not equally [V36]. |
| 2 | Why they can | Each ships its own C library [V9]. |
| 3 | So say different | Never say worse. |

What that trace cannot do

The honest caption, so nothing here is stronger than the evidence.

- 1 Not a matched pair** "It is not a matched pair." It follows the shell, not our program [V23].
- 2 Not what you will see** A 32-bit, XP-era trace, not today's addresses [V35].
- 3 One maintainer** And the layer it watches changes between releases [S9].
- 4 What survives** The handle, the path, the two error namespaces.

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

What we have not looked at

One question I will not answer, and the work that follows from it.

`write()` returned success. The bytes are not on a disk.

The whole hour built towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from write() does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question, and this lecture leaves it open.

[S2] write(2) — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984).
<https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Buffering moves your errors

The error arrives somewhere other than where you caused it.

- | | | |
|---|---------------------------|---|
| 1 | Why it moves | The bytes only reached a buffer. |
| 2 | Where it arrives | Where the library flushes, so at <code>fclose</code> . |
| 3 | The rule | Check the return value of the call that flushes. |
| 4 | The idiom hides it | "the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4]. |

Never retry a failed close

The obvious repair is wrong. Three storage engines dropped the retry in 2018 [V40].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

Three databases reached the same conclusion one level down, for fsync: PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger [V40]. WiredTiger wrote the reason into its own source comment: "we don't retry fsync-style calls and panic if a flush operation fails" [V24].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Networking and IPC are this same mechanism

Not new subjects. The same objects and the same calls, wearing different names.

1 **A socket on Linux** It is a file descriptor, and Linux makes that contract binding [V28].

2 **A socket on Windows** An opaque SOCKET, and Windows makes Winsock the contract [V28].

3 **A pipe** A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. The job here is to stop you filing networking under "something else".

Honest limits

What today's tools cannot see.

- **The device level** [V5]
Not observable from software: no trace can check it.
- **Asynchronous I/O** [V11]
The tracer sees the setup, then no I/O.

Honest limits, continued

And what today's tools cost.

- **strace in production**

[V25]

Even the cheaper `--seccomp-bpf` mode [V39] "should therefore not be used on production systems!"

- **Mapping, crash dumps**

Each has its own lecture.

What to do on Monday

- | | | |
|----------|--|---|
| 1 | Assignment 1 measures
takeaway 1 | Two settings, two patterns, one mebibyte. |
| 2 | Start from section one | Add the random-access variant. |
| 3 | Report your buffer size | Beside every number. |
| 4 | Do not benchmark under the
tracer | Count traced, time untraced [V17]. |
| 5 | Count in the dev container | Both platforms count there. |
| 6 | Week 2 opens | On 22 September. |

Summary — what follows

- | | | |
|---|---------------------------------|---|
| 1 | Ask "accepted or done?" | A fast return reports acceptance, not completion [V12]. |
| 2 | Count, do not reason | No level reports which one holds your data [V16]. |
| 3 | Name the level | Your buffer, the C library, the page cache, the device. |
| 4 | Measure your own machine | Never quote a remembered constant [S7]. |

References • 1/10

- **[S1]** strace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>
- **[S2]** write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html>
- **[S3]** fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- **[S4]** close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>
- **[S5]** C99 draft, §7.19.3. <https://busybox.net/~landley/c99-draft.html>
- **[S6]** Built-in Functions — open() — Python documentation.
<https://docs.python.org/3/library/functions.html#open>
- **[S7]** Increase io.DEFAULT_BUFFER_SIZE — CPython issue 117151.
<https://github.com/python/cpython/issues/117151>

References · 2/10

- **[S8]** PEP 3116 — New I/O (2007). <https://peps.python.org/pep-3116/>
- **[S9]** wininternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — Roger Orr. <https://rogerorr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S12]** Hints for Computer System Design — Lampson (1983). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- **[S13]** fsync — The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

References • 3/10

- **[S14]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S16]** FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf
- **[S17]** io.IOBase.flush() — Python documentation. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

References • 4/10

- **[S19]** Inside Native Applications — Russinovich, Microsoft Learn (2006).
<https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** os — Miscellaneous operating system interfaces, os.write() — Python documentation.
<https://docs.python.org/3/library/os.html#os.write>
- **[S21]** fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[V1]** A system call is not a function call — the two-term cost model.
- **[V2]** A system call is not a function call — the measured footprint of one crossing.

References · 5/10

- **[V3]** A system call is not a function call — pread and pwrite merge the seek.
- **[V4]** A Successful write() Is a Handoff, Not a Save — the idiom with no durability step.
- **[V5]** A Successful write() Is a Handoff, Not a Save — the device level is unobservable.
- **[V6]** Standard output is buffered differently depending on where it points — CPython above libc.
- **[V7]** Standard output is buffered differently depending on where it points — the measured 4096-byte buffer.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — the nomination argument.
- **[V9]** Windows and Linux place the stable syscall boundary at different layers — Linux as the outlier.

References · 6/10

- **[V10]** The System Call Boundary — ltrace against strace.
- **[V11]** The System Call Boundary — io_uring with a polling thread.
- **[V12]** Where the Abstraction Leaks — accepted, not done.
- **[V13]** Where the Abstraction Leaks — trust the counts, not the times.
- **[V14]** Where the Abstraction Leaks — a kill is not a power cut.
- **[V15]** Where the Abstraction Leaks — a privilege transition, not a jump.
- **[V16]** Where the Abstraction Leaks — count, do not deduce.
- **[V17]** Observing a Running System — the 442-times worst case.

References · 7/10

- **[V18]** The Memory Hierarchy from Python to Physical Pages — wholesale down, retail up.
- **[V19]** Virtual Memory and the Page Cache — name the cache level in every number.
- **[V20]** File IO and Durability — the device acknowledges early.
- **[V21]** NtTrace — Native API tracing for Windows (Roger Orr) — a debugger, not a driver.
- **[V22]** NtTrace — Native API tracing for Windows (Roger Orr) — the object-manager path prefix.
- **[V23]** NtTrace — Native API tracing for Windows (Roger Orr) — what the published trace cannot do.
- **[V24]** PostgreSQL's fsync() surprise — never retry after fsync fails.
- **[V25]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — not for production.

References · 8/10

- **[V26]** The System Call Boundary — the calls a tracer cannot see.
- **[V27]** A system call is not a function call — what merging the seek did to overall duration.
- **[V28]** Sockets and Network I-O — a Linux socket is a file descriptor, a Windows socket is not.
- **[V29]** Sockets and Network I-O — sockets, pipes and terminals are all file descriptors.
- **[V30]** File IO and Durability — block-layer tracing watches the kernel-to-device boundary.
- **[V31]** Observing a Running System — the published range for the same benchmark, on other machines.
- **[V32]** Virtual Memory and the Page Cache — the page cache is the machine's own memory.
- **[V33]** File IO and Durability — the kernel issues the device cache-flush command.

References · 9/10

- **[V34]** Virtual Memory and the Page Cache — readahead moves data up the same four levels.
- **[V35]** NtTrace — Native API tracing for Windows (Roger Orr) — the published trace is 32-bit and XP-era.
- **[V36]** Windows and Linux place the stable syscall boundary at different layers — the five systems do not decline the promise equally.
- **[V37]** File IO and Durability — FlushFileBuffers is the Windows counterpart of fsync.
- **[V38]** Windows and Linux place the stable syscall boundary at different layers — why Linux could not move its own boundary.
- **[V39]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — what the cheaper seccomp-bpf mode costs.

References • 10/10

- **[V40]** PostgreSQL's fsync() surprise — three engines dropped the retry in 2018.
- **[V41]** PostgreSQL's fsync() surprise — the three engines are independent of each other.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

TAKE IT WITH YOU

Everything from this talk

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

ondrasek.github.io/talks

me@ondrejkrasicek.com